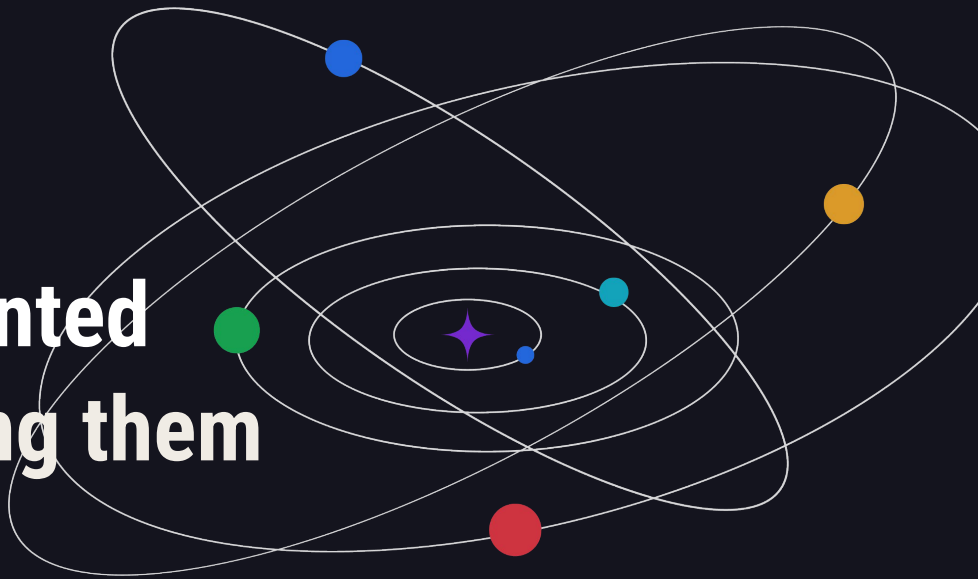


AIRFLOW SUMMIT 2026

Designing domain-oriented dbt projects and making them work in Airflow



Pankaj Koti Senior Software Engineer, Astronomer · Apache Airflow Committer & Cosmos maintainer

THE STORY IN ONE LINE

Your dbt project started at **20 models**.
Today it is **2,000**, and every team blocks every other team.

01

The problem

One monolith, many teams, why it stops scaling

02

The design

dbt Mesh: split into domain projects with clean interfaces

03

Make it run

Cosmos + Airflow orchestrate the pieces

One project. Every team. Every model.

- A single **dbt_project.yml** that finance, marketing, product and platform all commit to
- One `dbt run` builds the whole graph, staging, marts, everything, together
- It felt great at 50 models. It does not feel great at 2,000



How the monolith actually hurts

● Slow feedback

Parsing and CI grow with the whole graph, every change waits on everything

● Blurred ownership

Who owns this model? The whole company can edit it, so no one is accountable

● Blast radius

One broken staging model can stall marts for teams that never touched it

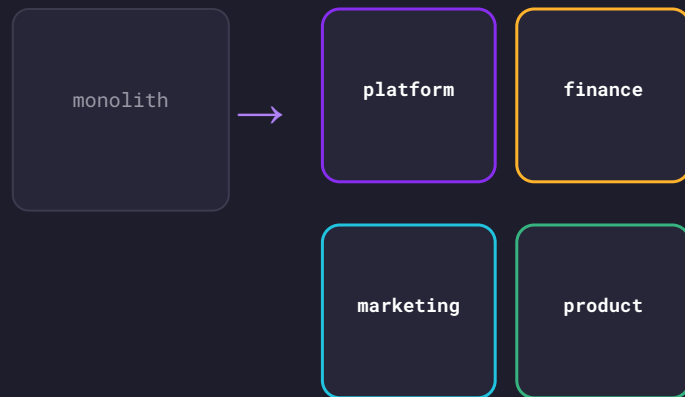
● Merge friction

Dozens of engineers, one repo, one YAML, constant conflicts

The fix: split by domain

Data mesh, applied to dbt. Give each **business domain** its own dbt project, owned by the team that lives in that data.

- **Platform** owns shared, foundational models
- **Finance, Marketing, Product** own their own marts
- Bounded contexts, clear edges, clear owners



A domain project has a public interface

- Owned by **one team**, in its own directory or repo
- Exposes a **small, deliberate set of public models**, its API
- Everything else stays **private**, free to refactor without breaking others
- Other domains depend on the **interface**, not the internals

platform project



private (staging, intermediate)



+access: public → the interface

finance / marketing consume only the public models

dbt Mesh is the toolbox for this

access

public / protected / private, define the interface

contracts

pin column names & types so consumers don't break

versions

evolve a model while old consumers still work

groups

assign ownership to a team

cross-project ref

reference another project's model by name

Cosmos leans on **access** and **cross-project ref** the most, that is the seam between projects.

The interface: **+access: public**

- Only **public** models can be referenced from another project
- **private** (default) and **protected** stay inside the domain
- This one line turns a model into a contract-bearing API surface

upstream/dbt_project.yml

```
models:
  upstream:
    staging:
      +materialized: view
      +access: public          # expose to others
    intermediate:
      +materialized: view
      +access: public

# everything else stays private
```

Governed change: contracts & versions

Contracts

Pin the column names and data types a public model promises.

A pre-flight check fails the build if the shape drifts, so a producer can't silently break a downstream consumer.

Versions

Ship v2 of a model while v1 still serves existing consumers.

Consumers migrate on their own schedule, coordinated change instead of a flag day.

The seam: reference, don't copy

- Declare the upstream project in **dependencies.yml**
- Then use **ref('project', 'model')** in your SQL
- One definition, one owner, no copy-paste of shared logic

downstream/dependencies.yml

```
projects:  
  - name: upstream
```

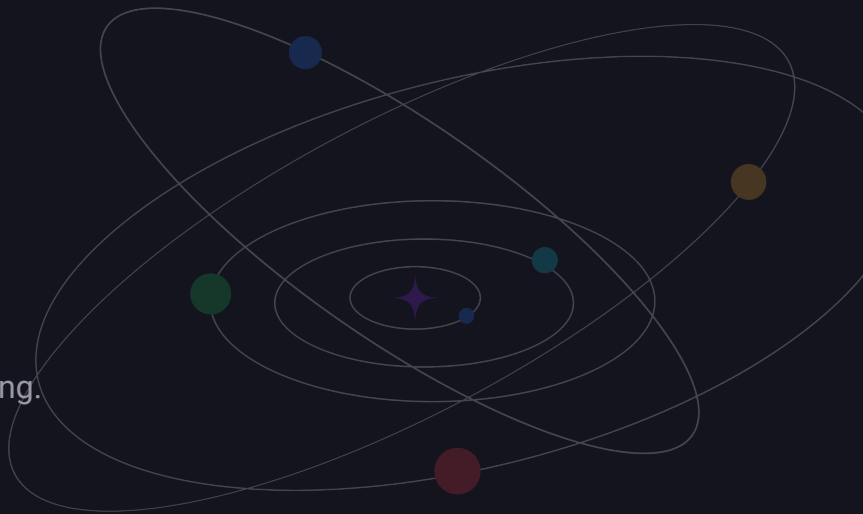
downstream/models/fct_revenue.sql

```
select * from  
  {{ ref('upstream', 'int_orders_enriched') }}  
where status = 'completed'
```

ACT 2 · THE ORCHESTRATION GAP

**You designed clean domains.
Now, who runs what,
in what order, across domains?**

A cross-project **ref()** is a logical link. It does not schedule anything.



Cross-project refs without dbt Cloud

- Native cross-project ref is a **dbt platform (Cloud)** feature
- On **dbt Core**, the open-source answer is **dbt-loom**
- A dbt Core plugin, no vendor lock-in, works with your own CI

`github.com/`

`nicholasyster/dbt-loom`

"Weaves together multi-project deployments."

`pip install dbt-loom`

How dbt-loom works

- 1 Reads the upstream project's **manifest.json**
- 2 Injects the upstream **public models** into the downstream project
- 3 Resolves **ref('upstream', 'model')** at parse time

dbt_loom.config.yml

```
manifests:  
  - name: upstream  
    type: file  
    config:  
      path: ../upstream/target/manifest.json  
    enable_telemetry: false
```

It is a dbt-core plugin (a **get_nodes** hook), the injection happens inside dbt itself.

The catch

dbt-loom **resolves references**. It does **not** orchestrate execution.

- The upstream tables must **exist** before downstream runs
- Something has to run the projects in **dependency order**
- That something is **Airflow**, and Cosmos is how dbt meets Airflow

Cosmos: dbt projects as Airflow DAGs

- Turns a dbt project into an Airflow **DAG or TaskGroup** automatically
- **One Airflow task per model**, not one opaque `dbt run`
- Model-level **retries, logs, and tests** in the Airflow UI
- Open source, maintained by Astronomer and the community

dbt project

↓ **Cosmos**

stg_customers.run

stg_orders.run

int_orders.run

fct_revenue.run

Cosmos in four configs

ProjectConfig

what to run, the dbt project or its manifest

RenderConfig

how it becomes a DAG, parsing, selectors, assets

ProfileConfig

credentials, Airflow connection → dbt profile

ExecutionConfig

how dbt runs, local, virtualenv, Docker, K8s

Cosmos + dbt-loom: it just works

- **Local nodes** (have a file path) → become Airflow tasks
- **External nodes** (injected by dbt-loom) → **skipped automatically**
- No special configuration, Cosmos detects them (fixed since 1.13)
- Each domain's DAG contains only that domain's own models

local model → task

local model → task

local model → task

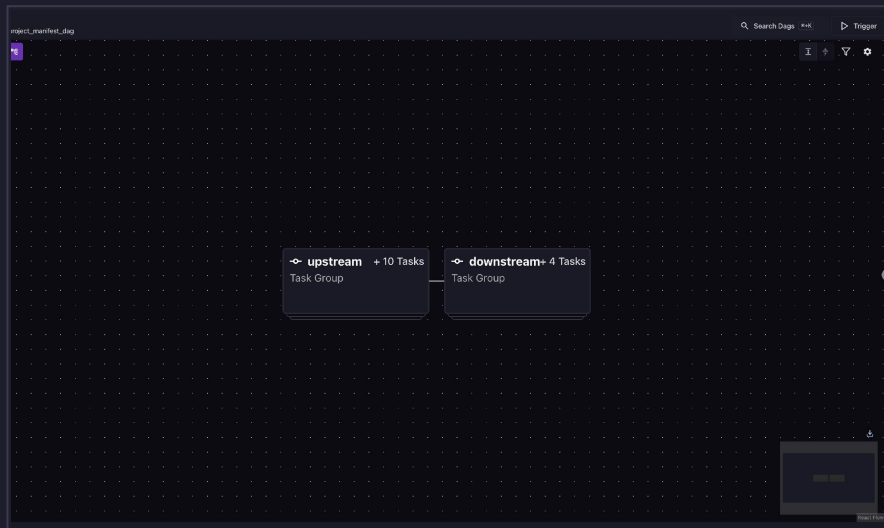
external (loom) → skipped

external (loom) → skipped

One DAG, a task group per domain

`cross_project_dbt_ls_dag.py`

```
upstream = DbtTaskGroup(  
    group_id="upstream",  
    project_config=ProjectConfig(UPSTREAM),  
    profile_config=platform, # schema: platform  
)  
downstream = DbtTaskGroup(  
    group_id="finance",  
    project_config=ProjectConfig(DOWNSTREAM),  
    profile_config=finance, # schema: finance  
)  
  
upstream >> downstream
```



Airflow graph, each domain is a collapsed TaskGroup

Isolation is a feature, not a workaround

- Each domain has its **own ProfileConfig**
- Different **schema** (platform vs finance)...
- ...or a different **database, warehouse, or credentials**
- Expand a group: **one task per model**, run + test

Expanded, model-level tasks inside each domain, all green

Faster parsing with manifest mode

- Parse from a **pre-built manifest.json**, no `dbt ls` at parse time
- Fastest DAG parsing; great for large projects
- Manifest can live in **S3, GCS or Azure** (manifest_conn_id)
- Generate manifests in **CI**, read them at runtime

cross_project_manifest_dag.py

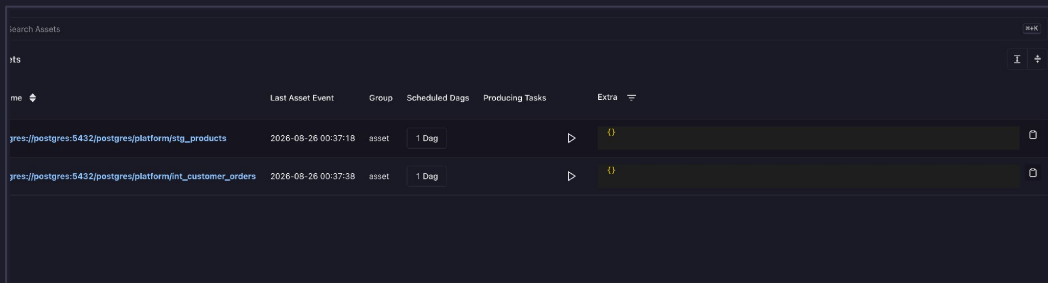
```
DbtTaskGroup(  
    group_id="upstream",  
    project_config=ProjectConfig(  
        manifest_path=str(UPSTREAM_MANIFEST),  
        project_name="upstream",  
    ),  
    render_config=RenderConfig(  
        load_method=LoadMode.DBT_MANIFEST,  
    ),  
)
```

Separate DAGs, wired by Assets

- Cosmos emits an **Asset** per model (OpenLineage URI)
- A downstream DAG is **scheduled on** those upstream assets
- Best when **different teams own different DAGs**
- Airflow 3 “Assets” = Airflow 2.4+ “Datasets”

downstream_dag.py

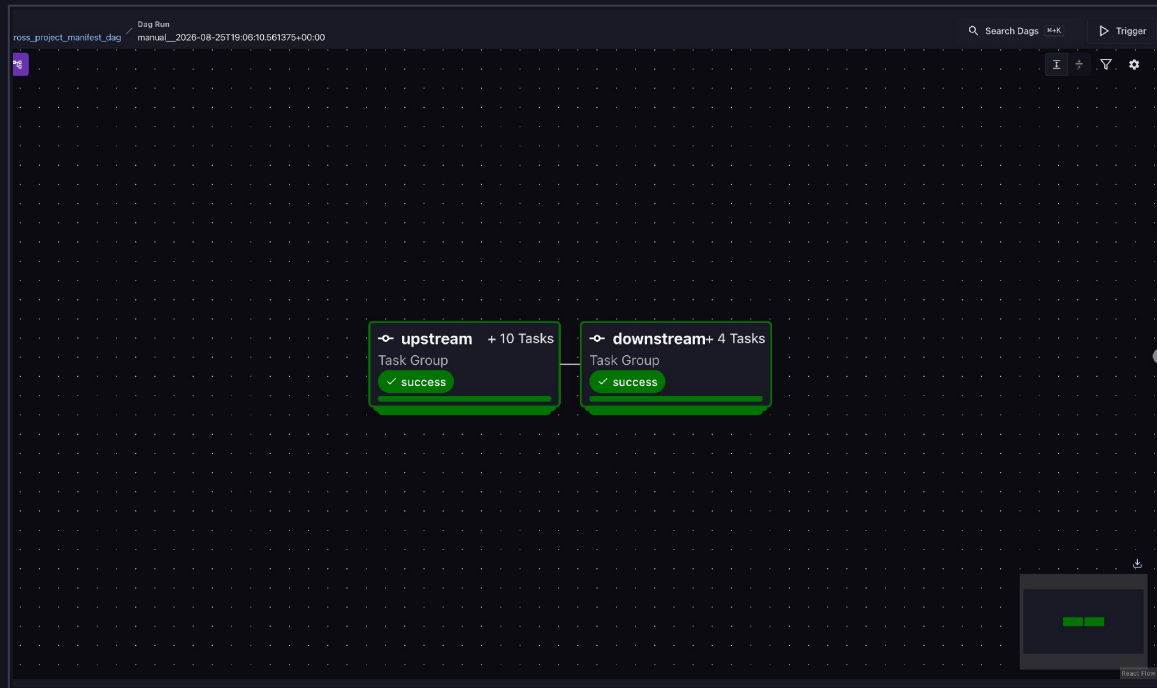
```
schedule=[UPSTREAM_ASSET],  
# runs when the upstream  
# model's asset updates
```



me	Last Asset Event	Group	Scheduled DAGs	Producing Tasks	Extra
gres://postgres:5432/postgres/platform/stg_products	2026-08-26 00:37:18	asset	1 Dag		
gres://postgres:5432/postgres/platform/int_customer_orders	2026-08-26 00:37:38	asset	1 Dag		

Assets Cosmos emits per model, each with a downstream DAG scheduled on it

Not a diagram, a real run



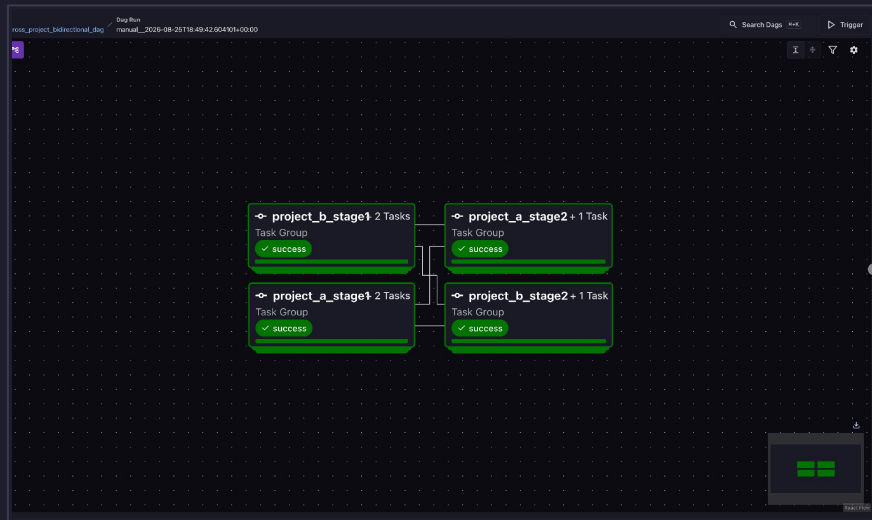
cross_project_manifest_dag • upstream (platform) → downstream (finance) • every model succeeded on Postgres via dbt-loom

Bidirectional refs: you own the ordering

- Two projects each expose a model the **other** consumes, a **diamond**, not a cycle
- Cosmos drops loom's external nodes, it does **not** wire cross-project deps
- So you split into **stage groups** and build the order by hand

```
cross_project_bidirectional_dag.py
```

```
[a_stage1, b_stage1] >> a_stage2  
[a_stage1, b_stage1] >> b_stage2
```



Diamond: both leaves build, then both reports, all green

Know the boundary

dbt-loom does

- Resolves cross-project **ref()**
- Injects upstream **public models**
- Runs inside dbt at **parse/run time**

Cosmos does

- Turns each project into **tasks**
- **Skips** injected external nodes
- Wiring cross-project **order is on you**

Gotchas worth knowing up front

● Cross-project sources

dbt-loom handles refs, not source(). Wrap a source in a public staging model.

● Shared macros

Not injected either. Put shared macros in a dbt package installed everywhere.

● Manifests must exist

Downstream parsing needs the upstream manifest present, generate it first.

● Warehouse identifiers

Manifests bake in the database name. Regenerate per target (Postgres vs BigQuery).

Practices that keep it sane

- Mark upstream models **+access: public** deliberately, it is your API
- Generate manifests in **CI**, store them where Airflow can read them
- Use **env vars** for manifest paths so it works across environments
- Order with **task-group chaining** (one DAG) or **assets** (separate DAGs)
- Use **persistent storage**, never in-memory DBs for cross-project data

Honest limitations today

- Cross-project **lineage isn't visualized** in Airflow yet
- A DAG can't emit **outlets** directly, use a completion-marker task
- Asset **URIs depend on connection details**, they change if those change
- Bidirectional setups need a careful **manifest bootstrap order** the first time

THE THROUGH-LINE

Monolith → domains → orchestrated

split by ownership · dbt Mesh + dbt-loom · Cosmos on Airflow

When to split

- Real team ownership boundaries
- Independent release cadences
- The monolith's CI actually hurts

When NOT to

- One team, one cadence
- Just a few hundred models
- Splitting to feel “modern”

THANK YOU

Take these with you



Cosmos docs, multi-project guide

astronomer.github.io/astronomer-cosmos



Example DAGs (dbt-loom cross-project)

github.com/astronomer/astronomer-cosmos → dev/dags



dbt-loom

github.com/nicholasager/dbt-loom



dbt Mesh

docs.getdbt.com, [best-practices](#) / [how-we-mesh](#)

Pankaj Koti · Astronomer, questions welcome

The 2026 Apache Airflow® Survey is Live!

- Takes just 7 minutes
- Get a free AI Orchestration Fundamentals Certification Exam or DAG Authoring Certification (normally \$150 each)



Thank you, Questions?

Pankaj Koti · Senior Software Engineer, Astronomer

Apache Airflow Committer & Cosmos maintainer

